FOGA: Flag Optimization with Genetic Algorithm

Burak Tağtekin
Huawei Turkey R&D Center
Istanbul, Turkey
burak.tagtekin1@huawei.com

Berkan Höke
Huawei Turkey R&D Center
Istanbul, Turkey
berkan.hoke1@huawei.com

Mert Kutay Sezer
Huawei Turkey R&D Center
Istanbul, Turkey
mert.kutay.sezer@huawei.com

Mahiye Uluyağmur Öztürk
Huawei Turkey R&D Center
Istanbul, Turkey
mahiye.uluyagmur.ozturk@huawei.com

2021 International Conference on INnovations in Intelligent SysTems and Applications (INISTA) | 978-1-6654-3603-8/21/$31.00 ©2021 IEEE | DOI: 10.1109/INISTA52262.2021.9548573

***Abstract*—Recently, program autotuning has become very popular especially in embedded systems, when we have limited resources such as computing power and memory where these systems run generally time-critical applications. Compiler optimization space gradually expands with the renewed compiler options and inclusion of new architectures. These advancements bring autotuning even more important position. In this paper, we introduced Flag Optimization with Genetic Algorithm (FOGA) as an autotuning solution for GCC flag optimization. FOGA has two main advantages over the other autotuning approaches: the first one is the hyperparameter tuning of the genetic algorithm (GA), the second one is the maximum iteration parameter to stop when no further improvement occurs. We demonstrated remarkable speedup in the execution time of C++ source codes with the help of optimization flags provided by FOGA when compared to the state of the art framework OpenTuner.**

***Keywords*—Compiler optimization, Compiler flags, Autotuning, Optimization, Genetic algorithm**

I. Introduction

Vast majority of software applications are developed using a high-level programming language such as C++, Java or C#. Specific applications that are designed to perform real-time, mission-critical operations or handle low-level memory management on the hardware level are mostly written in C/C++. Such C and C++ codes are passed to the compilers that translate these codes into binary executable files which can be interpreted by computer hardware. Compilers have been employed for 50 years in order to extract platform-specific binary executable from human-readable high-level programming languages [1], [2]. Compilers are still developed in terms of optimization so that each code block in a program can be translated into an optimized application. For tuning the compiler flags, some test cases are selected and the resulting performance is analyzed. Afterwards, the performance of the code is analyzed until it obtains the best flag set. Compilers includes three main layers and optimization can be applied to these layers distinctively: Frontend, Intermediate Representation, Backend. Optimizing source codes manually is also an option, but it is highly infeasible, since it is time consuming and probably platform-specific. Instead, compiler optimizations allow us to create optimized binary executable automatically. The scope of this study covers the optimization of intermediate representation of the compiler by enabling/disabling the corresponding optimization flags which yield to significant improvements based on the defined objectives. These objectives could be one of the followings but not limited to; power consumption, execution time (runtime), code size or compile time. The trade-off between these objectives yields to a challenging problem which requires holistic exploration [3].

Hence, an efficient approach should be employed in order to solve compiler optimization. Recent advances in artificial intelligence (AI) provides solution to seek optimization parameters using the previously known data. AI is the ideal fit for compiler optimization problem, due to following advantages:

- Once trained, a machine learning model can run fully automatic for previously unseen source code.
- Manually optimizing the compiler optimization flags is tedious task, where AI needs less intervention.
- Manual optimizations are usually valid for a single architecture or platform but an AI model could be architecture-independent.

Optimizing the compiler flags to run program faster is also referred as autotuning [4], [5]. In this study, we employ genetic algorithm approach in order to obtain the best compiler flags for execution time. GCC (GNU Compiler Collection) is the focus of our research for autotuning. Our search space contains 114 compilation flags from version 9.3 which implies that there are 2^{114} available combinations for binary flags. Among GCC optimization flags, namely **-O1**, **-O2**, **-O3**, **-Ofast**. -Ofast and -O3 optimization flags are referred as producing the most optimal flag sets. However, this is not the case for every C/C++ source code and compilation flags should be tweaked for some specific code blocks. To do so, machine learning models are utilized to solve search-space exploration and determining the best flag set. Pre-defined optimization levels of GCC is as follows:

- -O0
  This parameter does not make any optimization on the code, it only allows compiling the code. Main purpose here is to facilitate the debugging process.

978-1-6654-3603-8/21/$31.00 ©2021 IEEE

Authorized licensed use limited to: American Universtiy in Cairo - (AUC). Downloaded on November 12,2022 at 20:36:47 UTC from IEEE Xplore. Restrictions apply.

- -O1
  Enables some optimization flags to be compiled by optimizing the code by turning on some optimization flags. Here the compiler attempts to reduce the code size and execution time while keeping the compilation time as low as possible.
- -O2
  Optimize the code more than -O1. In this part, it is necessary to sacrifice memory for higher performance.
- -O3
  It aims to increase the optimization by turning on flags in -O2, as well as few other optimization flags.

This study provides a new approach to deal with automatic tuning of the C++ compilation flags by contributing the followings to existing methods:

- Seeking the optimum flag sets is dramatically low compared to other approaches.
- In order to find optimum runtime in a sensible time manner we use early stopping strategy.
- Hyperparameter fine-tuning of the genetic algorithm with evolutionary algorithms.

The remainder of the paper is organized as follows. First, a thorough literature survey on the problem is provided in Section II. Then, methods are introduced to predict compiler optimization flags for minimizing the execution time in Section III. Finally, experiments are provided followed by conclusions in Section IV and Section V respectively.

II. Related Work

Machine learning models have been used to solve problems from different areas and produces various outputs depending on the problem which includes seeking the best compiler flags or job scheduling and parallel program optimization [6]. In this section, we review the machine learning models used for compiler optimization. In general, machine learning model could be either supervised or unsupervised. Supervised learning expects labeled output from user and then reveals the relation between the input and the output such that this relation predicts the output label of the new data which is previously unseen by model. Depending on the prediction output, our supervised model could be either regression if the output has numerical features or classification if the output has categorical features. On the other hand, unsupervised methods do not need labels and mostly used for categorizing the input into several subsets. Recent advances in neural networks and available computing power, deep neural networks and online learning methods gain importance. Deep neural networks consists of many hidden layers where output of each layer is the weighted representation of the input layer depending on the layer type. They need more training data as they become more complex. Additionally, online learning approaches works like trial and error principle and tends to choose best model among all. In this chapter, different approaches of AI are reviewed in terms of compiler flag optimization.

A. Supervised Learning

Regression techniques are widely used for predicting numerical output. In the subject of compiler optimization, Qilin compiler [7] follows an approach so that it predicts the execution time of a program based on the input size for both CPU and GPU. Additionally, another proposed study uses regression techniques to estimate potential speedup of OpenCL programs with specific input [8]. Alongside estimating numerical features like execution time and speedup, classification techniques allow us to associate a program with classes implied from the input feature vectors.

Classification methods are often used for determining parameters. kNN is one of the simple yet effective classification approach used for choosing optimal optimization parameters in previously proposed studies [9], [10]. kNN algorithm chooses the best optimal flags from programs whose features are closest to the any given program. However, kNN has two major downsides: It gets slower in case of large training data. Secondly, kNN algorithm is not robust to noisy training data. Decision trees are alternative to kNN for classification tasks. For instance, loop unroll factors are determined by decision trees [11], [12], or optimal algorithms are selected for implementation [13]. Nevertheless, decision trees are prone to overfit training data. Random forests have been proposed to overcome overfitting problem [14]. Lokuciejewski et al. [15], proposed random forests to determine whether to inline a function or not . Also, support vector machines have been used for compiler optimization task in order to classify programs based on their feature vectors [16].

B. Unsupervised Learning

Unsupervised learning methods such as clustering automatically extract the target label of the input without any intervention instead of labels. Unsupervised learning generally used to reveal distribution of the data or categorize the data into the subgroups. In our specific objective for compiler optimization, works proposed in [17], [18] utilized k-means for clustering program execution path into phase groups. Besides k-means, fast Newman clustering approach [19] is utilized for categorize functions whose best optimization flags are similar to each other [20]. Unsupervised learning includes dimension reduction techniques as well. Principal component analysis(PCA) and autoencoders are such algorithms to reduce the dimension of feature space [21], [22]. PCA is often employed in order to alleviate the workload of compiler optimization techniques [23]–[25]. On the other hand, some methods have been proposed to extract representative programs from benchmarks [23], [26]. On the other hand, autoencoders are the artificial neural networks [22] which compress the features and reconstruct the input from this compressed representation. They are used to make source code feature more compact [27].

C. Online Learning

Evolutionary algorithms (EA) consists of three main categories: genetic algorithms (GA), genetic programming (GP) and stochastic-based search. All of them are utilized to find a

Authorized licensed use limited to: American Universtiy in Cairo - (AUC). Downloaded on November 12,2022 at 20:36:47 UTC from IEEE Xplore. Restrictions apply.

good optimization solution in very large search spaces. They work by mimicking the biological evolution to find the best fit for a given problem. Since EA is suitable for large search spaces, it is mostly used for seeking the best compiler flags in previously proposed works [28]–[30]. The algorithm starts with randomly initialized populations, where each population represents a compiler flag settings. These populations are employed to compile program with various optimization flag sequences and fitness function is generally defined as the execution time of the resulting program. Next population is derived from the current population via mechanisms such as reproduction/crossover and mutation. Compiler flags return lower execution times as these populations evolve by each iteration and algorithm returns the program with the lowest execution time. Agakov et al. [28] and Zuluaga et al [31] proposed to combine supervised learning and genetic algorithms by employing supervised model to narrow down the design space. Jantz and Kulkarni [32], proposed a method to prune design space. They pointed out that phase order search space is reduced by 89% on average.

III. Method

A. Genetic Algorithm

The concept of GA is inspired by the nature, weak species are faced with extinction by natural selection. The strong species are more successful in transferring their genes to subsequent generations via reproduction. There are parameters that affect the outcome of GA which are selection, crossover and mutation. Crossover, is the most time consuming part of genetic algorithm [17].

In this work, we represent compiler optimization flag sets as individuals and each gene of an individual in GA corresponds to a compiler optimization flag. Gene values are binary $\in \{0, 1\}$. If the flags are on, gene value is 1, otherwise 0. Phenotype representation is shown in Fig. 1.

As a fitness function, we try to optimize the execution time of source codes by choosing the most suitable compiler optimization flags. In other words, the fitness function returns a value proportional to the execution time of the target source code. Hence, if the execution time of the source code is longer, the fitness function will return a larger value or vice versa. Return value of a fitness function represents that individual's chance to survive within a given population (smaller value leads to higher survival ratio). The probability of survival is also related to the selection criteria. Stronger offsprings will be obtained by choosing strong parents for crossover. Considering this in our case, the genes that have little effect on reducing execution time are deleted from the population over time. The value of genes are defined by mostly crossover beside mutation. The mutation rate prevents us from sticking to the local minimas. In our problem, there are 2^{114} possible combinations for compiler optimization flags. The genetic algorithm allows us to quickly converge to a solution within such large search sets. Ultimately, when we do not observe any improvement, or when we realize that we do not get better offspring, the genetic algorithm stops and returns the best final individual as a result. Flowchart of the genetic algorithm is shown in Fig. 2.

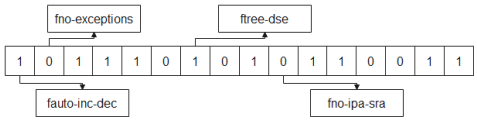


Fig. 1: Phenotype representation of compiler flags

B. Tuning Genetic Algorithm

Genetic algorithm has many parameters due to its structure. Population number, mutation rate, crossover type are a few of them. In order to achieve optimum solutions with GA, these parameters should be tuned in a smart way. In previous studies, [33] only four parameters were tuned on, but we did a very detailed study and wanted to run the genetic algorithm in the most accurate way. Available values for crossover, mutation and selection types are {*One Point, Two Point, Uniform, Shuffle, Segment*}, {*Gauss by Center, Uniform*}, {*Fully Random, Roulette, Sigma Scaling, Ranking, Linear Ranking, Tournament*} respectively. There are $5 \times 2 \times 7$ combinations for categorical variables and 7 different numerical parameters where our aim is tuning these parameters. We had experiments to iteratively compile and run the codes with these parameters. In order to do that, we used OpenTuner to find the best configurations by defining execution time of the best individual (proposed GA parameters of FOGA) as the optimization criteria.

As a result of the tuning process, we found that the best GA parameters are obtained which seeks compiler optimization flags as quick as possible. Tuned GA parameters are given

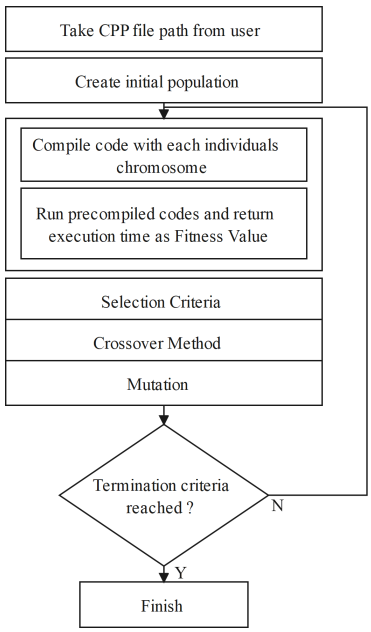


Fig. 2: Workflow of genetic algorithm for compiler optimization.

Authorized licensed use limited to: American Universtiy in Cairo - (AUC). Downloaded on November 12,2022 at 20:36:47 UTC from IEEE Xplore. Restrictions apply.

TABLE I: TUNED PARAMETERS

Max Number of Iteration	100
Population Size	277
Mutation Prob.	0.287
Elitism Ratio	0.147
Crossover Probability	0.120
Parents Portion	0.689
Crossover Type	Segment
Mutation Type	Gauss by Center
Selection Type	Linear Ranking
Max Iteration Without Improvement	45

in Table I. The best result was obtained through segment-based crossover, gauss by center mutation and linear ranking selection type. In the segment-based crossover, random chosen segments of the parents are used to form an offspring [34]. When compared to other crossover types, segment based crossover has noteworthy diversity on the gene. As a result, the search space contains various flag sets.

As mutation type gauss by center is used, where a random value which is driven from a normal distribution is added to the chosen gene [35].

In the linear ranking selection concept, the selection probability of an individual is determined by sorting fitness values with equal intervals between adjacent rank [36]. With linear ranking, individuals with lower fitness values have chance to be chosen. This approach provides to reveal the flags which has not been turned on but may have effect on the execution time.

IV. Experiments

Experiments are conducted to observe the runtime differences between autotuned optimization flags by OpenTuner and FOGA as well as the pre-defined optimization options (-O1, -O2, -O3). Three different source codes, namely *tsp_ga.cpp*, *matrixmutlipy.cpp*, *raytracer.cpp* are used where optimization plays significant role as these source codes mostly rely on linear algebra calculations. We used Ubuntu 20.04 with GCC version 9.3.0 for compiling C++ codes. Our experiments ran on Intel(R) Xeon(R) CPU E5 - 2670 v3 @ 2.30 GHz with 48 total cores.

We compiled the source codes with five different configurations; three of them are the pre-defined optimization options, the fourth configuration is the optimization flag set obtained by OpenTuner [37] and the last configuration is the flag set obtained from our proposed method FOGA.

In Fig. 3, runtime of the pre-defined optimization options, OpenTuner and FOGA are presented, x-axis shows the input size which is denoted as N and y-axis shows the runtime. Also, in Fig 4 the most important flags for each source code are shown. We performed one-hot encoding to understand effect of each flag to runtime. Flag importance is proportional to the corresponding speedup of specific flags. Their total speedup factor are rescaled to 100. Fig. 3a represents the execution time of the traveling salesman problem. Execution time of the pre-defined optimization levels are similar, while the execution time of OpenTuner and FOGA decreases as N

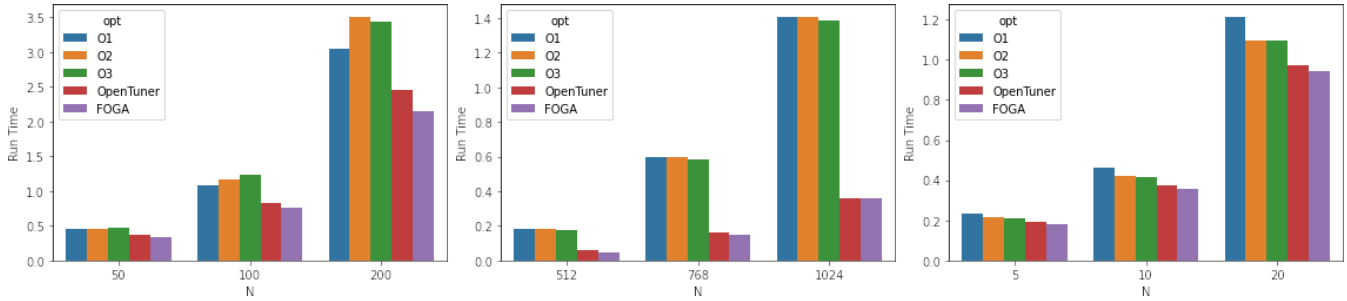
TABLE II: CONVERGENCE TIME COMPARISON BETWEEN FOGA AND OPENTUNER

Source Code	**OpenTuner(sec)**	**FOGA(sec)**
1d	124.47	66.47
2d	113.11	73.11
2mm	108.25	87.25
3d	80.21	68.21
3mm	87.29	50.29
adi	124.21	100.21
atax	120.55	67.55
bicg	125.91	63.91
cholesky	94.31	48.31
correlation	99.49	54.49
covariance	99.41	67.41
deriche	112.68	63.68
doitgen	108.82	50.82
durbin	100.93	45.93
gemm	91.57	35.57
gemver	96.16	125.16
gesummv	76.74	75.74
gramschmidt	90.12	34.12
lu	108.96	63.96
ludcmp	126.16	163.16
mvt	75.91	24.91
nussinov	118.38	75.38
symm	89.19	55.19
syr2k	74.25	43.25
syrk	113.03	55.03
trisolv	119.64	74.64
trmm	90.77	59.77
warshall	63.81	17.81

increases. This is an expected result, since according to Fig. 4a two most important flags, *funroll-all-loops* and *funsafe-math-optimizations* are not included in the default -O1, -O2 or -O3 GCC options. As it is seen in Fig. 3b, the margin between the pre-defined optimization levels and the evolutionary methods is wider compared to tsp_ga. Again, the most important flags are not included in -O1, -O2 or -O3. Moreover, these excluded *tree-loop-vectorize* and *unsafe-math-optimizations* flags are the most important ones for speedup, e.g. responsible for 41.54% and 40.70% of the speedup respectively as shown in Fig. 4b. Hence, we can deduce that FOGA and OpenTuner successfully find this relation and turn these flags on. On the third example, shown in Fig. 3c for *raytracer.cpp*, FOGA and OpenTuner slightly improves the execution time with respect to -O1, -O2, -O3. In this case, improvement ratio is lower, since importance of flags are similar and external flags (those not provided by the pre-defined optimization levels) does not improve the speedup so much for *raytracer.cpp*. In all cases, FOGA outperforms other configurations in terms of execution time regardless of the source code. Additionally, in Table II convergence time of FOGA and OpenTuner are represented based on PolyBench dataset [38] which consists of 28 source codes. Based on these convergence times, we conducted student's t-test and observed significant improvement where p-value is 9.07e-09 and confidence interval is 0.05.

In addition to the execution time of each source code, we also measured the duration of autotuning for reaching the best execution time for each source code. We compared the convergence time of FOGA and OpenTuner to shortest runtime

Authorized licensed use limited to: American Universtiy in Cairo - (AUC). Downloaded on November 12,2022 at 20:36:47 UTC from IEEE Xplore. Restrictions apply.



(a) tsp_ga.cpp (b) matrixmultiply.cpp (c) raytracer.cpp

Fig. 3: Comparison of speedup with compiler optimization flags.

Flags	Importance
funroll-all-loops	4.25%
funsafe-math -optimizations	1.92%
fno-tree-fre	1.37%
fno-inline -small-functions	1.37%
fno-tree-copy-prop	1.23%
111 other flags	89.86%

(a) tsp_ga.cpp

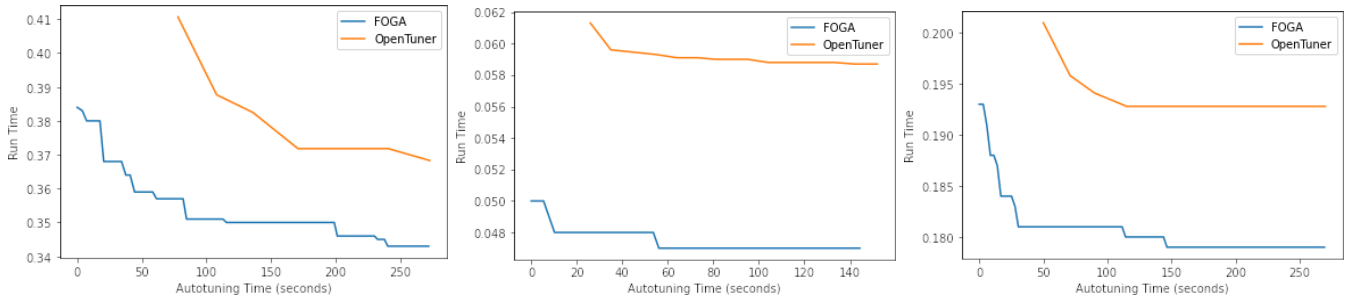
Flags	Importance
ftree-loop -vectorize	41.54%
funsafe-math -optimizations	40.79%
fno-tree-pta	0.47%
funroll-all-loops	0.39%
fno-dce	0.35%
111 other flags	16.46%

(b) matrixmultiply.cpp

Flags	Importance
fwrapv	5.31%
fno-unswitch-loops	1.64%
fno-forward-propagate	1.28%
fdelete-null- pointer-checks	1.28%
fgcse-lm	1.28%
111 other flags	89.21%

(c) raytracer.cpp

Fig. 4: Most important flag sets for each source code and their importance percentage values which are proportional to the corresponding speedup of specific flags.



(a) tsp_ga.cpp (b) matrixmultiply.cpp (c) raytracer.cpp

Fig. 5: Comparison of autotuning speedup for OpenTuner and FOGA.

as shown in Fig. 5. For all of the source codes, FOGA has remarkable advantage for both runtime and autotuning time. In Fig. 5b and Fig. 5c, FOGA stops searching whenever it does not provide any improvement after a certain period (see Table I). Note that OpenTuner continues to search for the best flag afterwards, however it could not reach better runtime than FOGA.

V. Conclusion

Making automatic adjustments at the compiler level is essential for a certain program to work optimally, especially for time-critical applications. With a smart automatic compiler parameter tuning method, it is aimed to increase performance by adjusting the compiler optimization flags. Sometimes turning off a compiler flag that is enabled by default increases the performance or in some cases activating a specific parameter enhances the efficiency. The mission of a autotuning method is to find the best possible parameters. In this study, we proposed FOGA which automatically tunes the compiler optimization flags with the help of genetic algorithm. As a first step, we tuned the parameters of genetic algorithm via OpenTuner by iteratively search among many combinations. Secondly, we showed that FOGA surpassed OpenTuner in terms of reaching the minimum execution time of a program. Additionally, we limited the iteration count if there is no improvement on the execution time for greater than a *Maximum Iteration without Improvement* parameter. These improvements allow us to reach shorter execution times in autotuning time with respect to the pre-defined optimization levels and OpenTuner.

Authorized licensed use limited to: American Universtiy in Cairo - (AUC). Downloaded on November 12,2022 at 20:36:47 UTC from IEEE Xplore. Restrictions apply.

As a future work, we plan to implement genetic algorithm that posses customized mutation and crossover operations. These customized operations are employed to keep alive important genes for the next generations. This approach narrows down the search space as individuals evolved and enable to find the best flags sooner.

References

[1] A. V. Aho, R. Sethi, and J. D. Ullman, "Compilers, principles, techniques," *Addison wesley*, vol. 7, no. 8, p. 9, 1986.

[2] M. Hall, D. Padua, and K. Pingali, "Compiler research: the next 50 years," *Communications of the ACM*, vol. 52, no. 2, pp. 60–67, 2009.

[3] G. Palermo, C. Silvano, and V. Zaccaria, "Multi-objective design space exploration of embedded systems," *Journal of Embedded Computing*, vol. 1, no. 3, pp. 305–316, 2005.

[4] K. Datta, M. Murphy, V. Volkov, S. Williams, J. Carter, L. Oliker, D. Patterson, J. Shalf, and K. Yelick, "Stencil computation optimization and auto-tuning on state-of-the-art multicore architectures," in *SC'08: Proceedings of the 2008 ACM/IEEE conference on Supercomputing*. IEEE, 2008, pp. 1–12.

[5] J. Ansel, Y. L. Wong, C. Chan, M. Olszewski, A. Edelman, and S. Amarasinghe, "Language and compiler support for auto-tuning variable-accuracy algorithms," in *International Symposium on Code Generation and Optimization (CGO 2011)*. IEEE, 2011, pp. 85–96.

[6] Z. Wang and M. O'Boyle, "Machine learning in compiler optimization," *Proceedings of the IEEE*, vol. 106, no. 11, pp. 1879–1901, 2018.

[7] C.-K. Luk, S. Hong, and H. Kim, "Qilin: exploiting parallelism on heterogeneous multiprocessors with adaptive mapping," in *2009 42nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2009, pp. 45–55.

[8] Y. Wen, Z. Wang, and M. F. O'boyle, "Smart multi-task scheduling for opencl programs on cpu/gpu heterogeneous platforms," in *2014 21st International conference on high performance computing (HiPC)*. IEEE, 2014, pp. 1–10.

[9] M. Stephenson and S. Amarasinghe, "Predicting unroll factors using supervised classification," in *International symposium on code generation and optimization*. IEEE, 2005, pp. 123–134.

[10] P.-J. Micolet, A. Smith, and C. Dubach, "A machine learning approach to mapping streaming workloads to dynamic multicore processors," in *Proceedings of the 17th ACM SIGPLAN/SIGBED Conference on Languages, Compilers, Tools, and Theory for Embedded Systems*, 2016, pp. 113–122.

[11] A. Monsifrot, F. Bodin, and R. Quiniou, "A machine learning approach to automatic production of compiler heuristics," in *International conference on artificial intelligence: methodology, systems, and applications*. Springer, 2002, pp. 41–50.

[12] H. Leather, E. Bonilla, and M. O'Boyle, "Automatic feature generation for machine learning based optimizing compilation," in *2009 International Symposium on Code Generation and Optimization*. IEEE, 2009, pp. 81–91.

[13] Y. Ding, J. Ansel, K. Veeramachaneni, X. Shen, U.-M. O'Reilly, and S. Amarasinghe, "Autotuning algorithmic choice for input sensitivity," *ACM SIGPLAN Notices*, vol. 50, no. 6, pp. 379–390, 2015.

[14] T. K. Ho, "Random decision forests," in *Proceedings of 3rd international conference on document analysis and recognition*, vol. 1. IEEE, 1995, pp. 278–282.

[15] P. Lokuciejewski, F. Gedikli, P. Marwedel, and K. Morik, "Automatic wcet reduction by machine learning based heuristics for function inlining," in *3rd workshop on statistical and machine learning approaches to architectures and compilation (SMART)*, 2009, pp. 1–15.

[16] P. Zhang, J. Fang, T. Tang, C. Yang, and Z. Wang, "Auto-tuning streamed applications on intel xeon phi," in *2018 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2018, pp. 515–525.

[17] E. Perelman, G. Hamerly, M. Van Biesbrouck, T. Sherwood, and B. Calder, "Using simpoint for accurate and efficient simulation," *ACM SIGMETRICS Performance Evaluation Review*, vol. 31, no. 1, pp. 318–319, 2003.

[18] T. Sherwood, E. Perelman, G. Hamerly, and B. Calder, "Automatically characterizing large scale program behavior," *ACM SIGPLAN Notices*, vol. 37, no. 10, pp. 45–57, 2002.

[19] M. Newman, *Networks: An Introduction*. New York, NY, USA: Oxford University Press, 2010.

[20] L. G. Martins, R. Nobre, A. C. Delbem, E. Marques, and J. M. Cardoso, "Exploration of compiler optimization sequences using clustering-based selection," in *Proceedings of the 2014 SIGPLAN/SIGBED conference on Languages, compilers and tools for embedded systems*, 2014, pp. 63–72.

[21] A. Magni, C. Dubach, and M. O'Boyle, "Automatic optimization of thread-coarsening for graphics processors," in *Proceedings of the 23rd international conference on Parallel architectures and compilation*, 2014, pp. 455–466.

[22] P. Vincent, H. Larochelle, Y. Bengio, and P.-A. Manzagol, "Extracting and composing robust features with denoising autoencoders," in *Proceedings of the 25th international conference on Machine learning*, 2008, pp. 1096–1103.

[23] L. Eeckhout, H. Vandierendonck, and K. De Bosschere, "Workload design: Selecting representative program-input pairs," in *Proceedings. International Conference on Parallel Architectures and Compilation Techniques*. IEEE, 2002, pp. 83–94.

[24] Y. Chen, Y. Huang, L. Eeckhout, G. Fursin, L. Peng, O. Temam, and C. Wu, "Evaluating iterative optimization across 1000 datasets," in *Proceedings of the 31st ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2010, pp. 448–459.

[25] A. H. Ashouri, G. Mariani, G. Palermo, and C. Silvano, "A bayesian network approach for compiler auto-tuning for embedded processors," in *2014 IEEE 12th Symposium on Embedded Systems for Real-time Multimedia (ESTIMedia)*. IEEE, 2014, pp. 90–97.

[26] A. Phansalkar, A. Joshi, and L. K. John, "Analysis of redundancy and application balance in the spec cpu2006 benchmark suite," in *Proceedings of the 34th annual International Symposium on Computer architecture*, 2007, pp. 412–423.

[27] X. Gu, H. Zhang, D. Zhang, and S. Kim, "Deep api learning," in *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 2016, pp. 631–642.

[28] F. Agakov, E. Bonilla, J. Cavazos, B. Franke, G. Fursin, M. F. O'Boyle, J. Thomson, M. Toussaint, and C. K. Williams, "Using machine learning to focus iterative optimization," in *International Symposium on Code Generation and Optimization (CGO'06)*. IEEE, 2006, pp. 11–pp.

[29] U. Garciarena and R. Santana, "Evolutionary optimization of compiler flag selection by learning and exploiting flags interactions," in *Proceedings of the 2016 on Genetic and Evolutionary Computation Conference Companion*, 2016, pp. 1159–1166.

[30] P. A. Ballal, H. Sarojadevi, and P. Harsha, "Compiler optimization: A genetic algorithm approach," *International Journal of Computer Applications*, vol. 112, no. 10, 2015.

[31] M. Zuluaga, E. Bonilla, and N. Topham, "Predicting best design trade-offs: A case study in processor customization," in *2012 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2012, pp. 1030–1035.

[32] M. R. Jantz and P. A. Kulkarni, "Exploiting phase inter-dependencies for faster iterative compiler optimization phase order searches," in *2013 International Conference on Compilers, Architecture and Synthesis for Embedded Systems (CASES)*. IEEE, 2013, pp. 1–10.

[33] T. Sandran, M. N. B. Zakaria, and A. J. Pal, "A genetic algorithm approach towards compiler flag selection based on compilation and execution duration," in *2012 International Conference on Computer & Information Science (ICCIS)*, vol. 1. IEEE, 2012, pp. 270–274.

[34] F. Altiparmak, M. Gen, L. Lin, and I. Karaoglan, "A steady-state genetic algorithm for multi-product supply chain network design," *Computers & Industrial Engineering*, vol. 56, no. 2, pp. 521–537, 2009.

[35] T. Bäck and H.-P. Schwefel, "An overview of evolutionary algorithms for parameter optimization," *Evolutionary computation*, vol. 1, no. 1, pp. 1–23, 1993.

[36] S. L. Yadav and A. Sohal, "Comparative study of different selection techniques in genetic algorithm," *Journal Homepage: http://www. ijesm. co. in*, vol. 6, no. 3, 2017.

[37] J. Ansel, S. Kamil, K. Veeramachaneni, J. Ragan-Kelley, J. Bosboom, U.-M. O'Reilly, and S. Amarasinghe, "Opentuner: An extensible framework for program autotuning," in *Proceedings of the 23rd international conference on Parallel architectures and compilation*, 2014, pp. 303–316.

[38] L. Pouchet, "The polyhedral benchmark suite," 2012, accessed: 2021-07-26. [Online]. Available: http://web.cse.ohio-state.edu/ pouchet.2/software/polybench/polybench.html

Authorized licensed use limited to: American Universtiy in Cairo - (AUC). Downloaded on November 12,2022 at 20:36:47 UTC from IEEE Xplore. Restrictions apply.